

1(a).Design a single unit perceptron for classification of a linearly separable binary dataset without using pre-defined models. Use the Perceptron() from sklearn.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron
from mlxtend.plotting import plot_decision_regions
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])
p = Perceptron()
p.fit(X, y)
print("Coefficients:", p.coef_)
print("Intercept:", p.intercept_)
z = p.score(X, y)
print("Accuracy score is:", z)
plot_decision_regions(X, y, clf=p)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Perceptron Decision Boundary')
plt.show()
```

1(b). Identify the problem with single unit Perceptron. Classify using OR, AND, XOR-ed data and analyze the result.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Perceptron
from mlxtend.plotting import plot_decision_regions
X_or = np.array([[0,0],[0,1],[1,0],[1,1]])
y_or = np.array([0,1,1,1])
X_and = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([0,0,0,1])
X_xor = np.array([[0,0],[0,1],[1,0],[1,1]])
y_xor = np.array([0,1,1,0])
def train_and_plot(X, y, title):
    p = Perceptron()
    p.fit(X, y)
    acc = p.score(X, y)
    print(f"{title} accuracy:", acc)
    plot_decision_regions(X, y, clf=p, legend=2)
```

```
plt.title(title)
plt.show()
train_and_plot(X_or, y_or, "OR Gate")
train_and_plot(X_and, y_and, "AND Gate")
train_and_plot(X_xor, y_xor, "XOR Gate")
```

2. Question

Build an Artificial Neural Network using Backpropagation and test using datasets. Vary activation functions and compare results.

Code:

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Dense, Activation

# Load Iris dataset
X, y = datasets.load_iris(return_X_y=True)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.40)

# Model
model = Sequential()
model.add(Dense(8, input_shape=(4,), activation='sigmoid'))
model.add(Dense(3, activation='softmax'))

# Compile
model.compile(loss='sparse_categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])

# Train
model.fit(X_train, y_train, epochs=50, batch_size=16)

# Evaluate
score = model.evaluate(X_test, y_test)
print("Test loss:", score[0])
print("Test accuracy:", score[1])
```

```
model.summary()
```

3 Design a Deep Feed Forward ANN by implementing Backpropagation algorithm and test using datasets. Vary number of hidden layers.

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import Dense

X, y = datasets.load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.40)

model = Sequential()
model.add(Dense(16, input_shape=(4,), activation='relu'))
model.add(Dense(12, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(6, activation='relu'))
model.add(Dense(3, activation='softmax'))

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=50, batch_size=16)

score = model.evaluate(X_test, y_test)
print("Test accuracy:", score[1])
model.summary()
```

4. QuestionDesign and implement an image classification model using Deep Feed Forward NN (MNIST).

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from sklearn.metrics import classification_report
import numpy as np
```

```

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax')])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.1)
y_pred = np.argmax(model.predict(X_test), axis=1)
print(classification_report(y_test, y_pred))
model.summary()

```

5. QuestionDesign and implement a CNN (2 layers) for image classification (MNIST).

```

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models
(train_X, train_y), (test_X, test_y) = datasets.mnist.load_data()
train_X = train_X.reshape(-1, 28, 28, 1) / 255.0
test_X = test_X.reshape(-1, 28, 28, 1) / 255.0
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_X, train_y, epochs=5, batch_size=64)
test_loss, test_acc = model.evaluate(test_X, test_y)
print("Accuracy:", test_acc)

```

6. Question Design a CNN with 4+ convolution layers for multi-category classification.

```
import tensorflow as tf
from tensorflow.keras import layers, models
(train_X, train_y), (test_X, test_y) = tf.keras.datasets.fashion_mnist.load_data()
train_X = train_X.reshape(-1,28,28,1)/255.0
test_X = test_X.reshape(-1,28,28,1)/255.0
model = models.Sequential([
    layers.Conv2D(128,(3,3),activation='relu',padding='same',input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu',padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu',padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(32,(3,3),activation='relu',padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_X, train_y, epochs=5, batch_size=64)
print(model.evaluate(test_X, test_y))
```

7. Question CNN with padding and Batch Normalization (Fashion MNIST).

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
model = models.Sequential([
    layers.Conv2D(32,(3,3),padding='same',activation='relu',input_shape=(28,28,1)),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
```

```

layers.Conv2D(64,(3,3),padding='same',activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D((2,2)),
layers.Flatten(),
layers.Dense(128,activation='relu'),
layers.Dense(10,activation='softmax'))])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, validation_data=(X_test,y_test))
loss, acc = model.evaluate(X_test, y_test)
print("Accuracy:", acc)

```

8. Design and implement a CNN model (with 4+ convolution layers) using regularization and dropout. Record training and test accuracy for:

- Base model
- Model with L1 Regularization

```

import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
import numpy as np
(train_X, train_y), (test_X, test_y) = tf.keras.datasets.fashion_mnist.load_data()
train_X = train_X.reshape(-1, 28, 28, 1) / 255.0
test_X = test_X.reshape(-1, 28, 28, 1) / 255.0
reg = regularizers.l2(0.01)
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', kernel_regularizer=reg,
input_shape=(28,28,1)),
    layers.Conv2D(32, (3,3), activation='relu', kernel_regularizer=reg),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3,3), activation='relu', kernel_regularizer=reg),
    layers.Conv2D(64, (3,3), activation='relu', kernel_regularizer=reg),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(128, activation='relu', kernel_regularizer=reg),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax'))])

```

```

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(train_X, train_y, epochs=5, batch_size=64,
                  validation_data=(test_X, test_y))
train_acc = history.history['accuracy'][-1]
test_loss, test_acc = model.evaluate(test_X, test_y)
print("Training Accuracy:", train_acc)
print("Test Accuracy:", test_acc)

```

9. Use data augmentation to increase dataset size from a single image.

```

import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
(X_train, _), _ = tf.keras.datasets.fashion_mnist.load_data()
img = X_train[0]
img = img.reshape(1, 28, 28, 1)
datagen = ImageDataGenerator(
    rotation_range=40,
    horizontal_flip=True,
    zoom_range=0.2
)
it = datagen.flow(img, batch_size=1)
plt.figure(figsize=(6,6))
for i in range(9):
    batch = next(it)
    plt.subplot(3,3,i+1)
    plt.imshow(batch[0].reshape(28,28), cmap='gray')
    plt.axis('off')
plt.show()

```

10. Implement RNN for sentiment analysis on movie reviews.

```

import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
max_features = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=max_features)
maxlen = 500
X_train = pad_sequences(X_train, maxlen=maxlen)

```

```

X_test = pad_sequences(X_test, maxlen=maxlen)
model = Sequential([
    Embedding(max_features, 32),
    SimpleRNN(32),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='rmsprop',
              loss='binary_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=3, batch_size=128, validation_split=0.2)
loss, acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", acc)

```

11 Implement Bidirectional LSTM for sentiment analysis on movie reviews.

```

import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, Dropout, Dense
max_features = 10000
maxlen = 200
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=max_features)
X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)
model = Sequential([
    Embedding(max_features, 128, input_length=maxlen),
    Bidirectional(LSTM(64)),
    Dropout(0.5),
    Dense(1, activation='sigmoid')])
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
history = model.fit(X_train, y_train, batch_size=128,
                    epochs=3, validation_data=(X_test, y_test))
print("Test Accuracy:", model.evaluate(X_test, y_test)[1])

```

12. Implement Autoencoders for image denoising on MNIST / Fashion MNIST.

```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential

```



```

from tensorflow.keras.layers import Dense, Flatten, Reshape
(X_train, _), (X_test, _) = tf.keras.datasets.mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0
X_train_noisy = X_train + 0.5 * np.random.normal(size=X_train.shape)
X_test_noisy = X_test + 0.5 * np.random.normal(size=X_test.shape)
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(64, activation='relu'),
    Dense(784, activation='sigmoid'),
    Reshape((28,28))])
model.compile(optimizer='adam', loss='mse')
model.fit(X_train_noisy, X_train, epochs=3)
decoded_imgs = model.predict(X_test_noisy)
plt.subplot(1,2,1)
plt.imshow(X_test_noisy[0], cmap='gray')
plt.title("Noisy")
plt.subplot(1,2,2)
plt.imshow(decoded_imgs[0], cmap='gray')
plt.title("Denoised")
plt.show()

```